

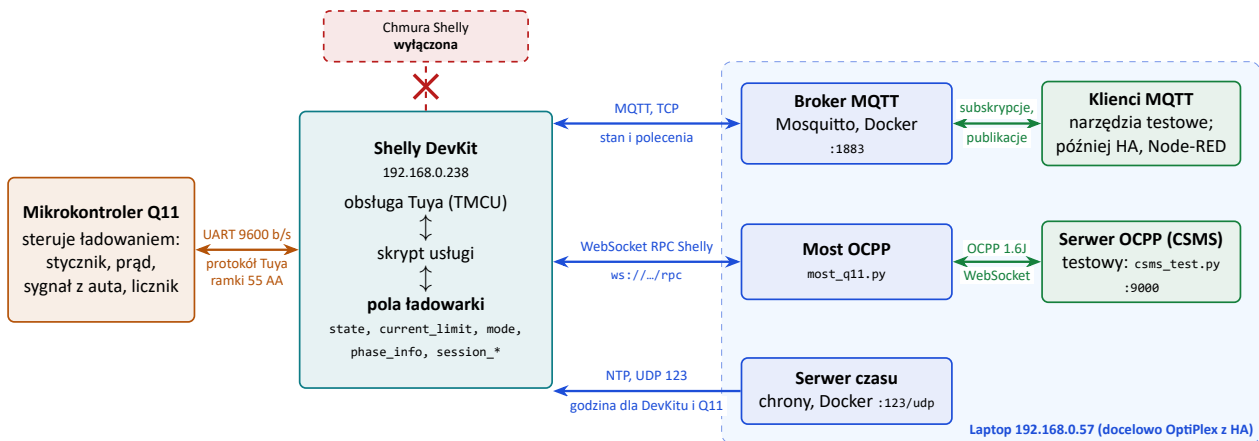
MQTT i OCPP w Q11 z modułem Shelly

Jak działają i jak je sprawdziliśmy · AMPERE POINT · 25 września 2026 · oprac. Dawid Cekała

W skrócie

- Moduł Shelly, na razie jako DevKit, siedzi w Q11 w miejscu modułu Tuya WBR3. Rozmawia z mikrokontrolerem ładowarki protokołem Tuya przy 9600 b/s.
- **MQTT**: DevKit sam wysyła stan ładowarki do brokera w sieci lokalnej i przez niego przyjmuje polecenia. Limit prądu wysłany przez MQTT Q11 potwierdziła w tej samej sekundzie.
- **OCPP 1.6J**: DevKit nie potrafi mówić OCPP sam. Robi to most, czyli program na komputerze, który tłumaczy OCPP na polecenia Shelly. Z serwerem testowym działa rejestracja, stan, pomiary, limit prądu i zdalny start.
- Oba kanały działają przy wyłączonej chmurze Shelly.
- Warunek pracy bez internetu to lokalny serwer czasu. Stoi w Dockerze na laptopie.
- Nie sprawdziliśmy jeszcze pełnej transakcji ładowania w OCPP. Potrzebne jest auto albo symulator pobierający prąd.
- Test był w sieci biura bez szyfrowania i hasła. W produkcji oba kanały muszą mieć TLS i uwierzytelnianie.

1 Układ całości



Rysunek 1: Połączenia w teście z 25.09.2026. Chmura Shelly była wyłączona, wszystko działało w sieci biura.

Element	Gdzie działa	Adres	Rola
Mikrokontroler Q11	płytką sterującą ładowarki	łącze szeregowo	steruje ładowaniem, mierzy, pilnuje bezpieczeństwa
Shelly DevKit	w obudowie Q11, zasilany z płytki	192.168.0.238	tłumaczy protokół Tuya na pola Shelly, wysyła MQTT, przyjmuje polecenia
Broker MQTT Mosquitto 2.1.2	Docker na laptopie	192.168.0.57:1883	przekazuje wiadomości między DevKitem a odbiorcami
Most OCPP most_q11.py	Python na laptopie	łączy się do DevKitu i serwera	udaje ładowarkę OCPP i przekłada polecenia na Shelly

Element	Gdzie działa	Adres	Rola
Serwer OCPP csms_test.py	Python na laptopie	ws://localhost:9000	udaje system operatora, zapisuje każdą wiadomość
Serwer czasu chrony	Docker na laptopie	192.168.0.57:123, UDP	podaje godzinę DevKitowi, a przez niego Q11

2 Warstwa pod spodem: DevKit i mikrokontroler Q11

MQTT i OCPP nie rozmawiają z Q11 bezpośrednio. Obsługują **poła** DevKitu, a skrypt w DevKicie przepisuje je na ramki protokołu Tuya i z powrotem. Dlatego najpierw ta warstwa.

2.1 Ramka protokołu Tuya

Bajty	Znaczenie
55 AA	początek ramki
1 bajt	wersja: moduł wysła 00, Q11 odpowiada 03
1 bajt	polecenie, np. 00 pytanie „jesteś?”, 06 zmiana punktu danych, 07 meldunek punktu danych, 1C prośba o godzinę
2 bajty	długość danych
dane	przy 06 i 07: numer punktu danych, typ, długość, wartość
1 bajt	suma kontrolna: suma wszystkich poprzednich bajtów, ostatni bajt wyniku

Typy wartości w danych: 00 surowe bajty, 01 tak lub nie, 02 liczba 4-bajtowa, 04 pozycja z listy, 05 mapa bitów.

Przykład

ustawienie limitu na 10 A to punkt danych 4, typ 02, długość 0004, wartość 0000000A. Cała ramka od modułu do Q11 według specyfikacji Tuya: 55 AA 00 06 00 08 04 02 00 04 00 00 00 0A 21. Suma kontrolna: $55+AA+00+06+00+08+04+02+00+04+00+00+0A = 121$ szesnastkowo, ostatni bajt to 21. Log DevKitu pokazuje sam fragment danych: TX: 0x06 [040200040000000a].

Odpowiedź Q11 w teście. Na każde polecenie zmiany Q11 odsyła dwa meldunki: najpierw starą wartość, a po około sekundzie nową. Widać to w logu przy każdej zmianie limitu:

```
11:56:48 TX: 0x06 [040200040000000a] moduł → Q11: ustaw punkt 4 na 10
11:56:49 RX: 0x07 [040200040000000c] Q11 → moduł: punkt 4 = 12 (jeszcze stara)
11:56:50 RX: 0x07 [040200040000000a] Q11 → moduł: punkt 4 = 10 (przyjęta)
```

2.2 Pola DevKitu, na których pracują MQTT i OCPP

Pole	Klucz w DevKicie	Źródło w Q11	Kierunek
włączenie ładowania, state	boolean:200	punkt danych 18	w obie strony

Pole	Klucz w DevKicie	Źródło w Q11	Kierunek
limit prądu, current_limit	number:200	punkt danych 4, 6–16 A	w obie strony
stan ładowarki, mode	enum:200	punkt danych 3	tylko z Q11
fazy i licznik, phase_info	object:200	punkty danych 6, 7, 8 i 1	tylko z Q11
czas sesji, session_duration	number:201	liczony przez skrypt	tylko odczyt
energia sesji, session_energy	number:202	liczona z punktu danych 1	tylko odczyt

Skrypt pilnuje, żeby wartość, która przyszła od Q11, nie wróciła do niej jako nowe polecenie. Bez tego każdy meldunek Q11 wywoływałby kolejną ramkę zmiany.

3 MQTT

3.1 Jak działa MQTT

MQTT to protokół wymiany krótkich wiadomości przez pośrednika, czyli **brokera**. Nikt nie łączy się z nikim bezpośrednio. Każdy uczestnik, czyli **klient**, łączy się tylko z brokerem:

- **publikuje** wiadomość na wybrany **temat**, np. `ampere/q11dev/status/number:200`,
- **subskrybuje** tematy, które go interesują; broker przekazuje mu każdą nową wiadomość z tych tematów.

Nadawca nie wie, kto słucha, a słuchaczy może być dowolnie wielu. Temat działa jak adres skrzynki, a znak # w subskrypcji oznacza „wszystko poniżej”.

Pojęcie	Znaczenie	U nas
broker	serwer, który przyjmuje i rozsyła wiadomości	Mosquitto w Dockerze, port 1883
temat	tekstowy adres wiadomości, poziomy rozdzielone /	przedrostek <code>ampere/q11dev</code>
QoS	poziom gwarancji doręczenia: 0 bez potwierdzenia, 1 co najmniej raz, 2 dokładnie raz	DevKit publikuje z QoS 1 według swojego logu
ostatnia wola, LWT	wiadomość, którą broker wyśle w imieniu klienta, gdy ten zniknie bez pożegnania	<code>ampere/q11dev/online = false</code>
utrzymanie połączenia	klient co jakiś czas daje znać, że żyje; brak sygnału oznacza zerwane połączenie	automatycznie

Dlaczego to wygodne dla ładowarki. DevKit wysyła stan raz, a korzystać z niego może jednocześnie Home Assistant, Node-RED, system zarządzania energią i własny serwer. Nowy odbiorca nie wymaga zmian w DevKicie.

3.2 Co wysyła DevKit

Temat	Kiedy	Zawartość
<code>ampere/q11dev/online</code>	przy połączeniu i przy zniknięciu	<code>true</code> albo <code>false</code>
<code>ampere/q11dev/status/<pole></code>	przy istotnej zmianie pola	pełny stan tego pola
<code>ampere/q11dev/events/rpc</code>	przy każdej zmianie	powiadomienie <code>NotifyStatus</code> ; po połączeniu pełny stan <code>NotifyFullStatus</code>
<code><src>/rpc</code>	po każdym poleceniu	odpowiedź na polecenie, na temat wskazany przez nadawcę

Fragment prawdziwego nasłuchu z 25.09 o 11:52, zaraz po włączeniu MQTT w DevKicie:

```
ampere/q11dev/online true
ampere/q11dev/status/boolean:200 {"value":true,"source":"sys","last_update_ts":1790328224}
ampere/q11dev/status/number:200 {"value":12,"source":"rpc","last_update_ts":1790326442}
ampere/q11dev/status/enum:200 {"value":"charger_free","source":"","last_update_ts":0}
ampere/q11dev/status/object:200 {"value":{"phase_a":{"voltage":0,"current":0,"power":0}, ...}}
```

```
ampere/q11dev/status/mqtt {"connected":true}
ampere/q11dev/events/rpc {"src":"apq11dev-543204516334","dst":"ampere/q11dev/events","method":"NotifyFullStatus", ...}
```

3.3 Jak wysła się polecenie

Polecenie to wiadomość JSON na temat `ampere/q11dev/rpc`. Ma ten sam format co polecenia wysyłane przez HTTP czy WebSocket. Pole `src` mówi DevKitowi, gdzie odesłać odpowiedź: na temat `<src>/rpc`.

```
temat: ampere/q11dev/rpc
treść: {"id":11,"src":"test/q11","method":"Number.Set","params":{"id":200,"value":10}}
```

```
odpowiedź trafia na temat test/q11/rpc
format odpowiedzi: {"id":11,"src":"apq11dev-543204516334","dst":"test/q11","result":...}
```

Inne przydatne polecenia: `Boolean.Set` z `"id":200` włącza albo wyłącza ładowanie, a `Number.GetStatus` czy `Enum.GetStatus` odczytują pole.

3.4 Ustawienia MQTT w DevKicie

Ustawienie	Wartość w teście	Znaczenie
enable	true	włącza klienta MQTT
server	192.168.0.57:1883	adres i port brokera
topic_prefix	ampere/q11dev	początek wszystkich tematów; domyślnie identyfikator urządzenia
client_id	apq11dev-543204516334	nazwa, pod którą broker widzi DevKit
rpc_ntf	true	powiadomienia o zmianach na <code>events/rpc</code>
status_ntf	true	pełny stan pól na <code>status/<pole></code>
enable_rpc	true	przyjmowanie poleceń na <code>ampere/q11dev/rpc</code>
enable_control	true	uproszczone tematy sterowania komponentami; w teście nieużywane
user, ssl_ca	puste	brak hasła i szyfrowania; dopuszczalne tylko w teście

Ustawienia zmienia polecenie `Mqtt.SetConfig` przez sieć lokalną, bez portalu. Zmiana wymaga restartu DevKitu, a DevKit sam o tym informuje: `"restart_required": true`.

3.5 Jak to sprawdziliśmy

1. **Broker.** Uruchomiłem kontener z oficjalnego obrazu `eclipse-mosquitto:2`, z konfiguracją: nasłuch na porcie 1883, dostęp bez hasła, bez zapisu wiadomości na dysk. Kontener startuje sam razem z Dockerem.
2. **DevKit.** Wysłałem mu przez HTTP polecenie `Mqtt.SetConfig` z ustawieniami z tabeli powyżej, a potem restart.
3. **Nasłuch.** Przez 45 sekund słuchałem wszystkich tematów, #, narzędziem `mosquitto_sub` wewnątrz kontenera. Przyszło 19 wiadomości: obecność, stan każdego pola i pełny stan urządzenia.
4. **Polecenie.** Narzędziem `mosquitto_pub` wysłałem na `ampere/q11dev/rpc` ustawienie limitu na 10 A, a potem powrót na 12 A.

5. **Sprawdzenie w Q11.** W logu DevKitu sprawdziłem, czy ramka poszła do Q11 i czy Q11 ją potwierdziła. Przebieg polecenia 10 A według logu DevKitu. Wszystko zmieściło się w jednej sekundzie:

Czas	Co się stało
11:53:06	DevKit dostaje przez MQTT polecenie <code>Number.Set</code> na 10 A
11:53:06	odpowiedź do nadawcy na temat <code>test/q11/rpc</code> , 70 bajtów
11:53:06	nowy stan pola na <code>ampere/q11dev/status/number:200</code> i powiadomienie na <code>events/rpc</code>
11:53:06	ramka do Q11: punkt danych 4 = 10
11:53:06	Q11 melduje najpierw starą wartość 12, zaraz potem nową 10
11:53:15–17	to samo dla powrotu na 12 A

Wynik: ✓. Stan przychodzi przez MQTT, a polecenie z MQTT dociera do Q11.

3.6 Ograniczenia MQTT w obecnej postaci

- **Brak zabezpieczeń.** Każdy w sieci biura może czytać tematy i wysyłać polecenia. Przed użyciem poza testem potrzebne jest konto z hasłem, szyfrowanie TLS i uprawnienia do tematów w brokerze.
- **Broker stoi na laptopie.** Gdy laptop jest wyłączony, MQTT nie działa. Docelowo broker ma działać na Opti-Plexie z HA.
- **Broker nie zapisuje wiadomości na dysk.** Po jego restarcie nowy odbiorca dostaje stan dopiero przy następnej zmianie albo przy ponownym połączeniu DevKitu.

4 OCPP 1.6J

4.1 Jak działa OCPP

OCPP to standardowy protokół rozmowy **ładowarki** z **systemem operatora**, nazywanym CSMS. Wersja 1.6J przesyła wiadomości w formacie JSON przez WebSocket. Najważniejsze zasady:

- **Połączenie nawiązuje ładowarka.** Łączy się z adresem serwera, dopisując na końcu swój identyfikator, np. `ws://localhost:9000/Q11-543204516334`. Przy łączeniu zgłasza podprotokół `ocpp1.6`.
- **Połączenie jest stałe.** Obie strony mogą w każdej chwili wysłać wiadomość, a druga strona musi na nią odpowiedzieć.
- **Każda wiadomość ma numer.** Odpowiedź niesie ten sam numer, więc wiadomo, na co odpowiada.

Trzy rodzaje wiadomości:

Rodzaj	Postać	Znaczenie
wywołanie	[2, "numer", "Akcja", {dane}]	prośba albo polecenie
wynik	[3, "numer", {dane}]	odpowiedź na wywołanie
błąd	[4, "numer", "kod", "opis", {szczegóły}]	wywołania nie da się obsłużyć

Prawdziwa wymiana z testu: ładowarka się przedstawia, serwer ją przyjmuje i każe dawać znak życia co 60 sekund.

```
→ [2, "aa145e23-...", "BootNotification", {"chargePointModel": "Q11", "chargePointVendor": "AmperePoint",  
"chargePointSerialNumber": "Q11-543204516334", "firmwareVersion": "Shelly X DevKit + skrypt v3"}]  
← [3, "aa145e23-...", {"currentTime": "2026-09-25T09:56:07Z", "interval": 60, "status": "Accepted"}]
```

4.2 Wiadomości OCPP i stan ich obsługi

✓ działa i sprawdzone, 🟡 zrobione, ale jeszcze niesprawdzone, ✗ nieobsługiwane.

Wiadomość	Kierunek	Po co	U nas
BootNotification	ładowarka → serwer	przedstawienie się po połączeniu	✓
Heartbeat	ładowarka → serwer	znak życia co ustalony czas	✓ co 60 s
StatusNotification	ładowarka → serwer	stan złącza	✓
MeterValues	ładowarka → serwer	pomiary	✓ na żądanie; 🟡 co 30 s w trakcie ładowania
StartTransaction	ładowarka → serwer	początek ładowania, licznik startowy, identyfikator karty	🟡
StopTransaction	ładowarka → serwer	koniec ładowania, licznik końcowy, powód	🟡
Authorize	ładowarka → serwer	sprawdzenie karty przed startem	✗ Q11 nie ma czytnika kart
RemoteStartTransaction	serwer → ładowarka	zdalny start z identyfikatorem karty	✓

Wiadomość	Kierunek	Po co	U nas
RemoteStopTransaction	serwer → ładowarka	zdalne zatrzymanie transakcji	🔴
SetChargingProfile	serwer → ładowarka	limit prądu albo mocy	✓
TriggerMessage	serwer → ładowarka	„wyślij teraz” stan, pomiary albo znak życia	✓
GetConfiguration	serwer → ładowarka	odczyt ustawień	✓
ChangeConfiguration	serwer → ładowarka	zmiana ustawień	🔴 tylko odstęp znaku życia
Reset	serwer → ładowarka	restart	✗ odrzucany
UnlockConnector	serwer → ładowarka	zwolnienie wtyczki	✗ Q11 ma kabel na stałe
pozostałe, np. rezerwacje i aktualizacje	serwer → ładowarka	—	✗ biblioteka odpowiada błędem „NotImplemented”

4.3 Stan złącza: z Q11 na OCPP

Stan Q11	Znaczenie	StatusNotification
charger_free	wolna, auto odłączone	Available
charger_insert	auto podłączone	Preparing
charger_wait	czeka, np. na harmonogram albo włączenie	SuspendedEVSE
charger_charging	ładuje	Charging
charger_pause	pauza	SuspendedEVSE
charger_end	zakończone	Finishing
charger_fault, charger_free_fault	błąd	Faulted, kod błędu OtherError

Most wysyła StatusNotification przy każdej zmianie stanu Q11 i na żądanie serwera.

4.4 Transakcja, czyli sesja ładowania w OCPP

1. Serwer wysyła **RemoteStartTransaction** z identyfikatorem karty. Most zapamiętuje identyfikator i włącza ładowanie w Q11, czyli pole state.
2. Gdy Q11 zgłosi stan „ładuje”, most wysyła **StartTransaction**: złącze 1, identyfikator karty albo Q11 - LOKALNIE przy starcie bez serwera, stan licznika w Wh i czas. Serwer odpowiada numerem transakcji.
3. W trakcie most co 30 s wysyła **MeterValues** z numerem transakcji.
4. **StopTransaction** most wysyła, gdy Q11 zgłosi koniec, odłączenie auta albo błąd, a także po RemoteStopTransaction. Powód to odpowiednio Local, EVDisconnected, Other albo Remote.

4.5 Limit prądu przez SetChargingProfile

Serwer przysyła profil ładowania. Most bierze z niego pierwszy okres harmonogramu i jego limit:

- limit w amperach zaokrągla do pełnych amperów,
- limit w watach przelicza na prąd fazy przez podzielenie przez 690, czyli 3 fazy × 230 V,
- wynik obcina do zakresu Q11, 6–16 A, i wpisuje w pole `current_limit`.

Przykład

11 000 W / 690 = 15,9, więc Q11 dostaje 16 A. 7 400 W / 690 = 10,7, więc 11 A. 3 000 W / 690 = 4,3, czyli poniżej minimum, więc 6 A.

Prawdziwy profil z testu, limit 10 A od zaraz, jako profil domyślny dla transakcji:

```
← [2,"60ca351b-...", "SetChargingProfile", {"connectorId":1, "csChargingProfiles":{"chargingProfileId":1,
"stackLevel":0, "chargingProfilePurpose":"TxDefaultProfile", "chargingProfileKind":"Absolute",
"chargingSchedule":{"chargingRateUnit":"A", "chargingSchedulePeriod":[{"startPeriod":0, "limit":10.0}
]}}}]]
→ [3,"60ca351b-...", {"status":"Accepted"}]
```

4.6 Pomiary w MeterValues

Wielkość OCPP	Jednostka	Skąd w Q11
Energy.Active.Import.Register	Wh	licznik całkowity, punkt danych 1
Power.Active.Import	W	suma mocy trzech faz, punkty danych 6–8
Current.Import, osobno L1, L2, L3	A	prąd fazy, punkty danych 6–8
Voltage, osobno L1, L2, L3	V	napięcie fazy, punkty danych 6–8
Current.Offered	A	limit prądu, punkt danych 4

4.7 Dlaczego potrzebny jest most

Werdykt: OCPP nie może dziś działać w samym module Shelly. OCPP 1.6J wymaga stałego połączenia WebSocket z dowolnym serwerem. Skrypt usługi w module ma dostęp do łącza z Q11, poleceń HTTP i zegara, ale nie ma klienta WebSocket. Wbudowany wychodzący WebSocket modułu mówi wyłącznie protokołem Shelly. Shelly nie udostępnia też narzędzi do dołączenia własnego kodu w C, na przykład gotowej biblioteki OCPP.

Most omija to ograniczenie: z modułem rozmawia protokołem Shelly, a z serwerem protokołem OCPP. Ten sam most może działać na dwa sposoby:

- **lokalnie**, jak w teście: most łączy się do DevKitu w sieci biura,
- **na serwerze**: DevKit sam łączy się z mostem wychodzącym WebSoketem, więc działa to także za routerem klienta, a jeden most obsługuje wiele ładowarek.

4.8 Jak działa nasz most

Strona DevKitu:

- łączy się z `ws://192.168.0.238/rpc` i wysyła polecenia Shelly z polem `"src":"most-ocpp"`; dzięki temu DevKit odsyła temu połączeniu powiadomienia o każdej zmianie pól,
- po połączeniu czyta pełny stan urządzenia, a potem osobno sześć pól ładowarki; pola usługi nie wchodzą do ogólnego stanu, co wyszło w teście,
- zmienia pola poleceniami `Boolean.Set` i `Number.Set`,
- jeśli DevKit nie zna godziny, ustawia mu ją poleceniem `Sys.SetTime`; to pomaga harmonogramom Shelly, ale nie godzinie podawanej Q11,
- po zerwaniu połączenia próbuje ponownie co 5 s.

Strona OCPP:

- korzysta z otwartej biblioteki `ocpp 2.1.0` dla Pythona i `websockets 17.1`,
- łączy się z `ws://localhost:9000/Q11-543204516334` z podprotokołem `ocpp1.6` i po zerwaniu próbuje ponownie co 10 s,
- po przyjęciu przez serwer wysyła stan, a potem znak życia w odstępie podanym przez serwer,
- nasłuchuje zmian stanu Q11 i na ich podstawie prowadzi transakcję.

Pełna droga polecenia limitu z serwera OCPP do Q11:

1. serwer OCPP wysyła `SetChargingProfile` przez WebSocket do mostu,
2. most wysyła do DevKitu `Number.Set` na pole `number:200`,
3. zmiana pola uruchamia w skrypcie usługi zapis punktu danych 4,
4. obsługa Tuya wysyła do Q11 ramkę zmiany,
5. Q11 przyjmuje wartość i melduje ją z powrotem,
6. DevKit rozsyła powiadomienie o nowej wartości: do mostu, przez MQTT i do aplikacji.

4.9 Serwer testowy

`csms_test.py` udaje system operatora. Przyjmuje każdą ładowarkę, odpowiada `Accepted` na rejestrację i karty, nadaje numery transakcji od 1 i zapisuje każdą wiadomość w `DevKit\logi\ocpp_csms_*.log`. Polecenia do ładowarki wpisuje się do pliku `DevKit\ocpp\polecenia.txt`, po jednym w wierszu: `start`, `stop`, `limit 10`, `status`, `meter`, `config`. Serwer sprawdza plik co sekundę.

4.10 Jak to sprawdziliśmy

Czas	Co się stało	Wynik
11:55:58	start serwera testowego na porcie 9000	✓
11:56:07	most łączy się i wysyła BootNotification; serwer: Accepted, znak życia co 60 s	✓
11:56:07	pierwszy StatusNotification: Available, ale z wartości domyślnej, bo most nie przeczytał pól	✗, błąd poprawiony
11:56:38	most po poprawce: odczyt stanu wolna, ładowanie włączone, limit 12 A; ponowna rejestracja	✓
11:56:48	TriggerMessage po pomiary: energia 0 Wh, moc 0 W, prąd i napięcie 0 na trzech fazach, oferowany prąd 10 A	✓
11:56:48	SetChargingProfile 10 A: Accepted; ramka do Q11 w tej samej sekundzie	✓
11:56:50	Q11 potwierdza 10 A	✓
11:56:57–59	SetChargingProfile 12 A: Accepted, Q11 potwierdza 12 A	✓
11:56:57	GetConfiguration: lista pięciu ustawień mostu	✓
11:57:20	RemoteStartTransaction z kartą KARTA-TEST: Accepted	✓
11:57:20	TriggerMessage po stan: Available	✓
11:57:20	polecenie zatrzymania: serwer go nie wysłał, bo nie było transakcji	zgodnie z oczekiwaniem
12:48:37–12:49:13	odłączenie zasilania ładowarki; most sam połączył się ponownie z DevKitem	✓
11:57–13:41	104 znaki życia, co 60 s, bez zerwania połączenia OCPP	✓

Pomiary z 11:56:48 pokazują zera, bo auto było odłączone, a stycznik otwarty. Napięcia faz Q11 zgłasza tylko przy zamkniętym styczniku.

4.11 Czego jeszcze nie sprawdziliśmy i co trzeba dodać

1. **Pełna transakcja:** start, pomiary co 30 s i koniec. Potrzebne jest auto albo symulator pobierający prąd.
2. **Licznik całkowity podaje 0 kWh.** Transakcja miałaby wtedy licznik startowy i końcowy równy zero. Trzeba sprawdzić z wyświetlaczem ładowarki, czy ten egzemplarz rzeczywiście ma zero.
3. **Kody błędów.** Dziś każdy błąd Q11 idzie jako OtherError. Potrzebne jest pole z mapą błędów Q11, czyli punkt danych 10, żeby wysyłać konkretne kody, np. GroundFailure.
4. **Kolejka offline.** Gdy serwer OCPP jest niedostępny, most nie przechowuje zdarzeń. Transakcja rozpoczęta w tym czasie nie trafi do serwera.
5. **Stan transakcji w pamięci mostu.** Restart mostu w trakcie ładowania gubi numer transakcji.
6. **Bezpieczeństwo.** Test szedł przez ws:// bez hasła. OCPP 1.6 przewiduje profile bezpieczeństwa: 1 hasło, 2 hasło i TLS, 3 certyfikaty obu stron. W produkcji minimum profil 2.
7. **Most na stałe.** Dziś działa na laptopie. Docelowo jako usługa na OptiPlexie albo na serwerze AmperePoint.

5 Czas: warunek wspólny dla obu kanałów

Q11 regularnie prosi moduł o godzinę, poleceniem 1C, i na niej opiera harmonogram ładowania. OCPP stempluje czasem każdy pomiar i każdą transakcję. W teście bez serwera czasu moduł odpowiadał Q11 ramką „czas ważny” z samymi zerami. Ręczne ustawienie godziny w DevKicie tego nie zmieniał, bo obsługa Tuya bierze godzinę tylko z serwera czasu.

```
bez serwera czasu: TX: 0x1c [0100000000000000] „ważny”, ale rok 0, godzina 0
z lokalnym serwerem czasu: TX: 0x1c [011a09190c320c05] 2026-09-25 12:50:12, piątek
```

Dlatego na laptopie działa serwer czasu chrony w Dockerze, a DevKit ma go wpisane jako jedyne źródło. Bez internetu chrony podaje czas zegara komputera, zamiast milczeć. Po restarcie DevKit zsynchronizował się w 18 s.

6 Gdzie co jest zapamiętane

Co	Gdzie	Po restarcie
ustawienia MQTT, serwer czasu, wyłączona chmura	DevKit, pamięć flash	zostają
limit prądu, włączenie ładowania	DevKit i Q11	zostają; Q11 i tak podaje swoje wartości po starcie
numer transakcji OCPP i identyfikator karty	pamięć mostu	giną przy restarcie mostu
numeracja transakcji w serwerze testowym	pamięć serwera	zaczyna się od 1
wiadomości MQTT	broker, bez zapisu na dysk	nie są przechowywane
licznik energii, nastawy i harmonogram ładowania	mikrokontroler Q11	zostają w Q11
zapisy testów	DevKit\logi	pliki ocpp_csms_*, ocpp_most_*, mqtt_pierwszy_nasluch.txt

Słowniczek

Pojęcie	Znaczenie
broker MQTT	serwer, który przyjmuje wiadomości od klientów i rozsyła je subskrybentom
temat MQTT	tekstowy adres wiadomości, np. ampere/q11dev/rpc
CSMS	system operatora, z którym ładowarka rozmawia w OCPP
identyfikator karty, idTag	tekst identyfikujący użytkownika w transakcji OCPP
transakcja OCPP	jedna sesja ładowania, od StartTransaction do StopTransaction
profil ładowania	harmonogram limitów prądu albo mocy wysyłany przez serwer

Pojęcie	Znaczenie
WebSocket	stałe połączenie sieciowe, przez które obie strony mogą wysyłać wiadomości w dowolnej chwili
punkt danych	numerowana wartość w protokole Tuya, np. 4 = limit prądu
pole DevKitu	wartość w module Shelly, na której pracują aplikacja, MQTT i most, np. number:200
polecenie RPC	wywołanie funkcji DevKitu, np. Number.Set, przez HTTP, WebSocket albo MQTT

Załącznik: pliki i polecenia

Co	Gdzie
most OCPP	SHELLY\DevKit\ocpp\most_q11.py
serwer testowy OCPP	SHELLY\DevKit\ocpp\csms_test.py, polecenia w polecenia.txt
konfiguracja brokera	SHELLY\DevKit\mqtt\config\mosquitto.conf
serwer czasu	SHELLY\DevKit\ntp\Dockerfile i chrony.conf
skrypt usługi w DevKicie	SHELLY\DevKit\portal_q11\script.svc.ts
logi testów	SHELLY\DevKit\logi

Uruchomienie OCPP, w dwóch oknach wiersza poleceń:

```
python -u SHELLY\DevKit\ocpp\csms_test.py
python -u SHELLY\DevKit\ocpp\most_q11.py 192.168.0.238 ws://localhost:9000
```

Podgląd MQTT i wysłanie polecenia, z kontenera brokera:

```
docker exec mosquitto mosquitto_sub -h localhost -t "ampere/q11dev/#" -v
docker exec mosquitto mosquitto_pub -h localhost -t ampere/q11dev/rpc -m '{"id":1,"src":"test/q11",
"method":"Number.Set","params":{"id":200,"value":10}}'
```